R GIS Developer Tools Open Source Data Science

pukRhelpers: A Personal Toolbox of R Helpers

Small utilities for ArcGIS services, UGRC data, KML and sf that I reused across projects

An R package collecting helper functions from my day-to-day data and GIS work. Maintained lightly.

Pukar Bhandari 2025-05-23

Table of contents

1 Install **1**
2 What is in it **2**
3 Try one in your browser **2**

`pukRhelpers` is where the small functions I kept copying between projects ended up. It is a personal toolbox, not a polished library, and I only update it when my own work needs something. The package has continuous checks and test coverage set up, and documentation at ar-puuk.github.io/pukRhelpers.

Docs GitHub

1 Install

```
pak::pak("ar-puuk/pukRhelpers")
install.packages("pukRhelpers", repos = "https://ar-puuk.r-universe.dev")
```

Line 1
Straight from GitHub. `remotes::install_github()` and `devtools::install_github()` work too.

Line 2
Or from my personal R-universe.

2 What is in it

Function	What it does
`load_esri()`	Load and query an ESRI Feature Service layer
`arc_read_encode()`	`arcgislayers::arc_read()` with coded-value encoding built in, so no stray helper objects are left in the environment
`load_ugrc_data()`, `load_ugrc_vars()`	Load a dataset, or list the datasets, from Utah's SGID
`load_kml_sf()`	Convert KML to an `sf` object with attributes preserved
`st_aggregate()`	Aggregate `sf` objects
`geo_split_lines()`, `st_split_lines()`	Split lines by a maximum length
`convert_fgb_to_gdb()`	Convert FlatGeobuf files to geodatabase layers
`convert_xml_html()`, `extract_html_table()`	XML to HTML via XSLT, and pull a table out of an HTML page by its anchor
`load_packages()`	Load, installing first if needed

Several of these build on other people's gists and are credited to them in the function documentation and the README.

3 Try one in your browser

The package itself cannot run inside a web page: it talks to ArcGIS services and databases, and it needs system libraries that browsers do not have.

`st_aggregate()` is different, because it is plain `sf` and base R. The code below is copied from the package and run on the North Carolina counties that ship with `sf`.

First, load the counties and sort them into three zones:

`st_aggregate()` comes from a gist by rCarto, and the package credits it the same way.

Then define the function. It joins the geometries of each zone with `st_union()` and summarises the attributes you ask for:

Now aggregate: the mean and median births per zone, with the counties merged into three shapes. Try changing `"BIR74"` to `"SID74"`, or `"mean"` to `"max"`:

```
library(sf)

nc ← st_read(system.file("shape/nc.shp", package =
"sf"), quiet = TRUE)
nc$zone ← "ZONE_A"
nc$zone[25:50] ← "ZONE_B"
nc$zone[51:100] ← "ZONE_C"

st_aggregate ← function(x, by, var, fun) {
  var ← c(by, var)
  fun ← c("head", fun)
  n ← length(var)
  l ← vector("list", n)
  for (i in 1:n) {
    l[[i]] ← tapply(x[[var[i]]], x[[by]],
add_args(fun[[i]]), n = 1, na.rm = TRUE)
  }
  names(l) ← var
  st_sf(do.call(data.frame, l),
        geometry = tapply(x[attr(x, "sf_column")],
x[[by]], st_union),
        crs = st_crs(x))
}

add_args ← function(x) {
  rx ← get(x)
  fx ← formals(rx)
  fx$na.rm ← TRUE
  fx$n ← 1
  formals(rx) ← fx
  rx
}

zones ← st_aggregate(nc, "zone", c("BIR74",
```

```
"NWBIR74"), c("mean", "median"))
plot(zones["BIR74"], main = "Mean births per county,
1974")
```

The web version of this post runs these steps live in the browser.

The code is on GitHub under GPL-3.